

Cougar Planner

Kenneth Wilkins, Amanda Chaaban, Daniel Zapata, Maria

Grassa, Manal Asad

Team 1

SE 471: Final Project Report

Table of Contents

1 Introduction.....	3
1.1 Project background and description.....	3
1.2 Key features implemented.....	3
1.3 Key users.....	4
1.4 Secondary users.....	4
2 System Design.....	4
2.1 Command Pattern.....	4
2.2 Filter Pattern.....	6
2.3 Cache Pattern.....	7
2.4 GUI Tasks.....	8
2.5 GUI Settings.....	9
3 System Implementation.....	10
3.1 Command Pattern.....	10
Command Manager.....	10
Command Interface.....	11
Concrete Command Example: CreateTaskCommand.....	11
GUI Visuals.....	12
3.2 Filter Pattern.....	13
3.3 Cache Pattern.....	17
3.4 GUI Tasks.....	17
3.5 GUI Settings.....	18
5 Lessons Learned.....	20
5.1 Kenny.....	20
5.2 Amanda.....	21
5.3 Maria.....	21
5.4 Manal.....	22
5.5 Daniel.....	22
Works Cited.....	23

1 Introduction

1.1 Project background and description

Cougar Planner is a JavaFX-based application made for CSUSM students that integrates with the Canvas API to access information. Using an API token that the user must acquire themselves, the system is able to get courses, assignments, and announcements, then stores this data locally in structured CSV files along with user preferences in a .profile file, which allows for full offline functionality after initializing data. The application has a simple and easy to understand GUI with a week based navigation, sorting, and toggling between assignments and announcements, while also having features that allow the user to configure the week start and API token saving. The system prioritizes usability, reliability, maintainability, and also allows a fallback to view saved data when offline or API issues happen.

1.2 Key features implemented

Our team has identified several areas that have room for improvement in Cougar Planner. Firstly, the naming of certain portions of the GUI, as well as the functionality, should be revised so that they are accurate, functional, and intuitive for the user. For example, “Hello user!, and welcome to the Cougar Planner!!!” should be revised to be grammatically correct. Also, “Orientation” should be renamed “Instructions for obtaining API token”. Furthermore, it currently states, “*List API token acquisition instructions”, without listing the instructions and shall be revised to list them. Next, “API token” shall be renamed “login”, and “show” shall be renamed “display”. Lastly, the following buttons on the GUI are currently non-functional including sort by, show, week start, and settings, and shall become functional. Secondly, there is presently a shadow that remains on the calendar after the user moves their cursor away from viewing announcements, and they must click elsewhere on the calendar to remove it. We would like to clean up this visual to remove that shadow. Third, we would like to provide the user with the option to customize the theme of their Cougar Planner by choosing one of three options that we will provide. Lastly, we initially planned on, but never implemented, a feature to provide the user with the ability to rate their assignments with a level of difficulty which would then determine the length of time to dedicate to said

assignment. However, our team has determined that it would be a better fit for our project to instead add a task tab feature, where the user may add their own events, tasks, or subtasks. Moreover, we plan to provide the user with the ability to either check and uncheck, or delete tasks with the option of deleting them either individually, or all at once with a delete all option. Ultimately, correcting these defects, implementing these new features, and enhancing performance will greatly improve our project and we have determined that the three best design patterns to achieve this include Filter, Cache Management, and Command.

1.3 Key users

Users that are actively enrolled in courses at CSUSM.

- **User role responsibilities:** Complete the orientation for Cougar Planner and download any applicable software if they do not already have it.
- **Subject matter experience:** The user may be considered a novice and new to CSUSM and navigating Cougar Courses.
- **Technological experience:** The user may be a novice, but must have version Java 17 or newer downloaded on their computer, and be able to authenticate themselves with a token as indicated in the orientation process of Cougar Planner.
- **Other user characteristics:** Should be able to read from their device, physically hold their device, and have the dexterity to click, scroll, and use a standard GUI interface.

1.4 Secondary users

A caregiver or student aid if the student has any impairments that prevent them from using Cougar Planner.

2 System Design

2.1 Command Pattern

The Command Pattern encapsulates a request as an object, allowing us to create many different commands as well as support undo and redo operations. The architectural overview is that the

design decouples the object that invokes the operation (the GUI) from the one that knows how to perform it (the CommandManager). This is achieved through our different components:

- **Command Interface:** A common interface that declares an undefined execute() method.
- **Concrete Commands:** Classes like CreateTaskCommand, DeleteTaskCommand, and UpdateTaskCommand that implement the interface and define the binding between a receiver and an action.
- **Receiver:** the TaskRepository, which performs the actual logic, such as writing and removing from the CSV file.
- **Invoker:** The Task Tab GUI component that triggers the command's execution.

Using this pattern allows for the system to stay decoupled while supporting maintainability and scalability as the project grows. It addresses separation of concerns by ensuring that changes to the task logic do not require modification to the GUI code. Since this also satisfies the Open/Closed Principle, we can simply create a new Concrete Command class without changing any existing command or invoker code.

By implementing the Command Pattern, Cougar Planner remains organized, flexible, and easier to extend as new features and task operations are added over time.

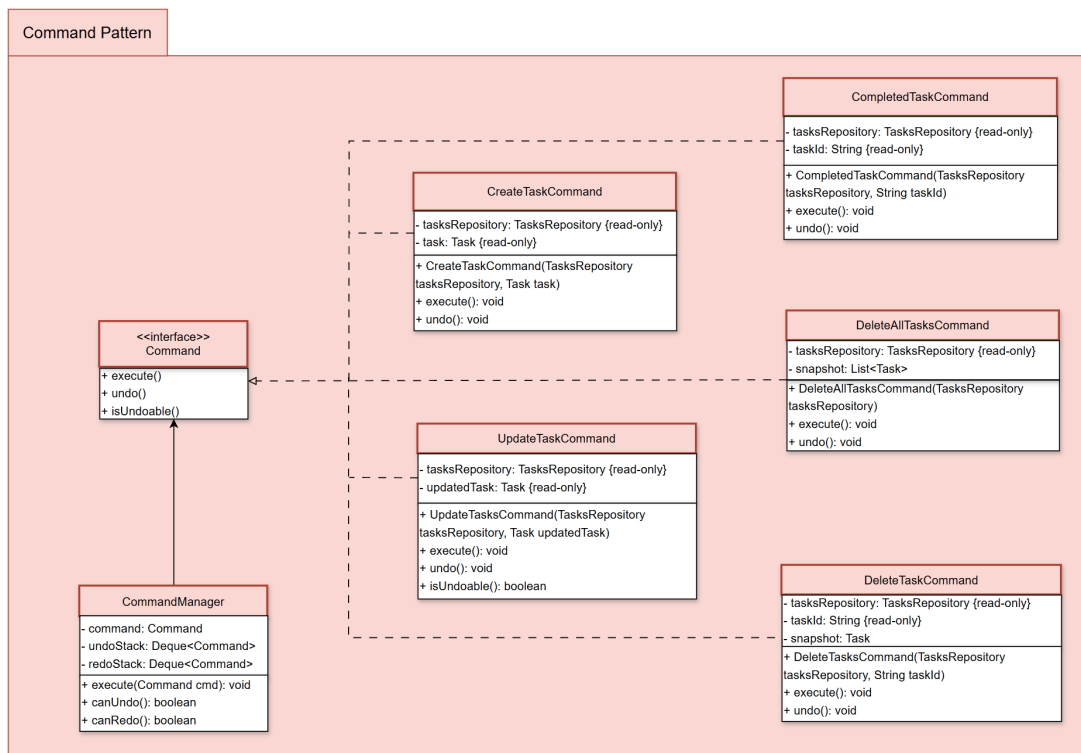


Figure 2.1a: Command Pattern UML class diagram

2.2 Filter Pattern

The filter pattern is a structural design pattern that enables filtering a set of objects using different criteria. These criteria can also be combined logically using AND/OR operations without coupling the filtering logic to the client code. In Cougar Planner, this pattern is used to filter tasks based on various attributes such as course, priority, status, and due date. This helps users to narrow down their task list based on these specific criteria. The pattern is designed to be flexible so new filters can be added later without changing existing code. The core components for this pattern are as follows:

- **TaskFilter Interface:** This is the foundation of the pattern. It declares a single method *filter(list<Task> tasks)* that takes a list of tasks and returns a filtered list. Every filter we create must implement this interface.
- **Concrete Filters:** Then we have four concrete filters that implement the interface.
 - CourseFilter: shows tasks from one specific course
 - PriorityFilter: shows tasks with high, medium, or low priority
 - StatusFilter: shows tasks that are Not Started, in progress, or completed
 - DueDateFilter: shows tasks due that are selected by the user
- **ANDFilter (composite):** the ANDFilter holds a list of multiple filters. When its *filter()* method is called, it applies every filter one by one. This allows users to combine criteria. For example, “SE-471”, AND “High Priority”, AND “Not Started”.

Basically, how it works is that the user selects filters in the UI. The FilterBarController is responsible for creating individual filters. The filters are added to ANDFilter, and then ANDFilter applies all filters chosen by the user sequentially. Finally, the TaskPanelController receives the filtered results and displays only matching tasks.

The design benefit is that it is open for extension or is scalable. We can add new filters without changing existing filter code. Each filter works independently and can be used anywhere. Moreover, ANDFilter lets users combine multiple filters naturally.

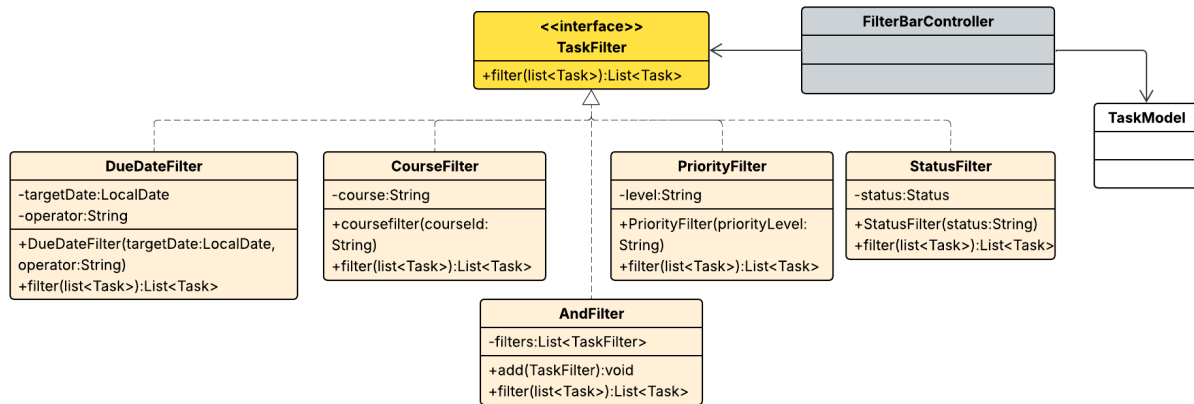


Figure 2.2a: Filter pattern UML class diagram, showing filter interface and concrete filters, along with controller and model

2.3 Cache Pattern

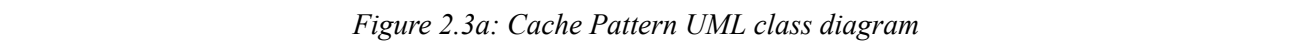
The Cache Pattern was created to reduce I/O operations during session runtime. Without this pattern, every navigation action, going between different weeks, would trigger a CSV file read, even if the data had just been loaded previously. This pattern works by using a 3-layer data access point:

Layer 1 - LRU Memory Cache: Holds recently accessed data in memory for instant retrieval with no disk or network access.

Layer 2 - CSV Repository: The existing file layer is retrieved only when memory cache misses, which provides full offline functionality.

Layer 3 - Canvas API - The network layer that is reached when neither the memory cache nor the CSV repository holds the requested data.

The design is based on four parts. The `Cache<T>` interface defines methods (`get`, `put`, `remove`, `size`) that all cache implementations must fulfill. `LinkedListLRUCache<T>` implements this interface using a doubly linked list paired with a `HashMap`. `CacheManager` is a singleton that owns a named registry of caches (courses, assignments, announcements). The three concrete cache subclasses (`CourseCache`, `AssignmentCache`, `AnnouncementCache`) extend `LinkedListLRUCache` to allow future type customization.



to its own FXML and controller pair, allowing the panel to scale to any number of tasks while remaining independently maintainable.

2.5 GUI Settings

The settings panel relies on a few ways to keep the settings panel feeling responsive and keeping the settings persistent across sessions of usage. When the panel opens, `bindMainController` links it to the `MainPageController` and reads the live status of every label. So, the panel will always get the most current up to date information and can avoid being mismatched with what the user sees. Each option group follows the same “radio button” like styling, so stay consistent with the rest of the program (Like the tasks panel). The controller tracks the active labels so that the theme switcher can load the appropriate CSS files when needed to ensure the correct theme is applied. On selection, the click handler calls back into main controller through `setDisplayView`, `setWeekStart`, or `setTheme`, which changes the in memory profile and immediately re-renders the affected views/planner sections. Theme changes specifically route through `ThemeManager.resolve` and `applyToAllOpenWindows` so that every open stage restyles in place. Persistence works because every setter writes the profile to disk through `ProfileWriter` right after it mutates state, which is why preferences survive across sessions. Closing the panel via the x button calls `mainController.closeSettingsPanel`, which hides the pane and falls back to whichever top-level view — Announcements, Assignments, or Tasks — the sort group is currently highlighting.

3 System Implementation

3.1 Command Pattern

The Command Pattern is built into how the tasks feature works. Anytime the user creates a task, deletes one, marks it complete, or clears the whole list, the action goes through the `CommandManager` instead of the GUI, directly touching the CSV file. Each action is wrapped in its own command class, and the Task Tab just calls `cmdManager.execute(new CreateTaskCommand(...))` when the user clicks on a button. The `CommandManager` runs the command's `execute()` method and stores it in the undo stack. Undo simply pops the last command and calls `undo()` on it, and redo works the same way with its own stack.

This setup keeps `TaskPanelController` clean because it never handles file logic. All the work happens inside the command classes, which talk to the `TaskRepository` to update the CSV. After a command finishes, the `TaskCache` is updated so the GUI always shows the latest state without the extra file reads. Adding new task actions is also easy. You would just make a new command class and plug it in without changing anything else. This keeps the system organized, consistent, and ready for more features later.

Command Manager

```
public class CommandManager {  
    private final Deque<Command> undoStack = new ArrayDeque<>();  
    private final Deque<Command> redoStack = new ArrayDeque<>();  
  
    public void execute(Command cmd) {  
        try {  
            cmd.execute();  
            if (cmd.isUndoable()) {  
                undoStack.push(cmd);  
                // A new action invalidates the redo history — the user has  
                // branched off from the previous redo timeline.  
                redoStack.clear();  
            }  
        } catch (Exception ex) {  
            throw new RuntimeException("Command execution failed", ex);  
        }  
    }  
}
```

```
public void undo() {  
    if (undoStack.isEmpty()) return;  
    Command cmd = undoStack.pop();  
    try {  
        cmd.undo();  
        redoStack.push(cmd);  
    } catch (Exception ex) {  
        throw new RuntimeException("Command undo failed", ex);  
    }  
}
```

```
public void redo() {  
    if (redoStack.isEmpty()) return;  
    Command cmd = redoStack.pop();  
    try {  
        cmd.execute();  
        undoStack.push(cmd);  
    } catch (Exception ex) {  
        throw new RuntimeException("Command redo failed", ex);  
    }  
}  
public boolean canUndo() { return !undoStack.isEmpty(); }  
public boolean canRedo() { return !redoStack.isEmpty(); }  
}
```

Command Interface

```
public interface Command {  
    void execute() throws Exception;  
  
    void undo() throws Exception;  
  
    // In-row field edits (title/description) are excluded from the undo stack  
    // to avoid character-by-character history. Commands that shouldn't be  
    // recorded override this to return false.  
    default boolean isUndoable() { return true; }  
}
```

Concrete Command Example: CreateTaskCommand

```
public class CreateTaskCommand implements Command {  
    private final TasksRepository tasksRepository;  
    private final Task task;
```

```

public CreateTaskCommand(TasksRepository tasksRepository, Task task) {
    this.tasksRepository = tasksRepository;
    this.task = task;
}

```

```

@Override
public void execute() throws Exception {
    TaskCache.getInstance().add(task);
    tasksRepository.upsert(task);
}

```

```

@Override
public void undo() throws Exception {
    TaskCache.getInstance().remove(task.getTaskId());
    tasksRepository.deleteById(task.getTaskId());
}
}

```

GUI Visuals

The screenshot shows a mobile application interface. At the top, there's a dark blue header with 'Viewing Tasks' on the left and 'Settings' on the right. Below the header, there's a 'Tasks' tab selected, with 'Courses' and 'All Courses' tabs also visible. The main content area is a list of tasks. Overlaid on this is a 'Add Task' form. The form has a title field with 'Finish Project report', a due date field, a description field with 'Task description', and three dropdown menus for 'Course' (SE 471), 'Status' (not started), and 'Priority' (high). At the bottom of the form are 'Cancel' and 'Add Task' buttons. At the very bottom of the screen is a navigation bar with 'Undo', 'Redo', 'Add Task', and 'Delete All' buttons.

Figure 3.1a: The form to create a task

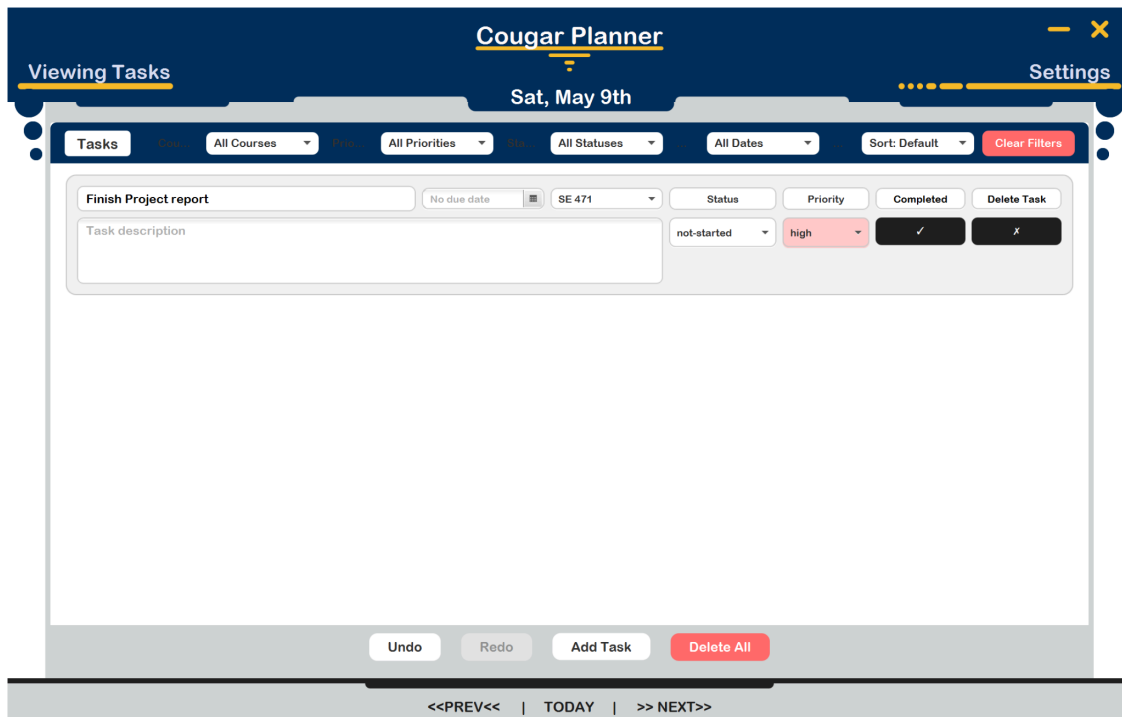


Figure 3.1b: Updated list of tasks with a new task that was created

3.2 Filter Pattern

The filter pattern was implemented using an interface-based approach. First, we create a `TaskFilter` interface with a single `filter()` method that takes a list of tasks and returns a filtered list.

```
public interface TaskFilter {
    List<Task> filter(List<Task> tasks); }
```

Then, we built four concrete filter classes: `CourseFilter`, `PriorityFilter`, `StatusFilter`, and `DueDateFilter`. Each of these concrete filters implements the `TaskFilter` interface and handles one specific filtering criterion.

CourseFilter:

@Override

```
public List<Task> filter(List<Task> tasks) {
    if (courseId == null || courseId.isEmpty()) {
        return tasks;
    }
    return tasks.stream()
        .filter(task -> courseId.equals(task.getCourseId()))
```

```
        .collect(Collectors.toList());  
    }  
}
```

DueDateFilter:

@Override

```
public List<Task> filter(List<Task> tasks) {  
    return tasks.stream()  
        .filter(task -> {  
            if (task.getDueDate() == null) return false;  
            LocalDate dueDate = LocalDate.parse(task.getDueDate(), FORMATTER);  
            switch (operator) {  
                case "eq": return dueDate.equals(targetDate);  
                case "before": return dueDate.isBefore(targetDate);  
                case "after": return dueDate.isAfter(targetDate);  
                default: return false;  
            }  
        })  
        .collect(Collectors.toList());  
}
```

PriorityFilter:

@Override

```
public List<Task> filter(List<Task> tasks) {  
    return tasks.stream()  
        .filter(task -> priorityLevel.equals(task.getPriority()))  
        .collect(Collectors.toList());  
}
```

StatusFilter:

@Override

```
public List<Task> filter(List<Task> tasks) {  
    return tasks.stream()  
        .filter(task -> status.equals(task.getStatus()))  
        .collect(Collectors.toList());  
}
```

ANDFilter:*@Override*

```
public List<Task> filter(List<Task> tasks) {  
    List<Task> result = tasks;  
    for (TaskFilter filter : filters) {  
        result = filter.filter(result);  
    }  
    return result;  
}
```

The key challenge was combining multiple filters together, which we solved with an ANDFilter class that also implements the interface but internally holds a list of other filters and applies them one after another. For UI integration, FilterBarController loads courses from the CourseRepository, populates dropdown menus, and builds the appropriate filters based on user selections. When a user selects filter options, the controller creates individual filter objects, adds them to an ANDFilter, and passes it to the TaskPanelController. Then TaskPanelController calls filter() on whatever filter it receives, whether a single filter or a combined one, and refreshes the task list display. For date filtering, we also had to handle a specific date format("MM-dd-yyyy") using a custom DateTimeFormatter because the tasks stored dates differently than Java's default. Moreover, the sorting functionality was added separately after filtering, using a simple comparator to order tasks by due date, from earliest to latest or from latest to earliest. Overall, the implementation was straightforward once the interface was defined, and adding a new filter type would only require creating a new class that implements the interface, and no changes were needed to existing filter code.

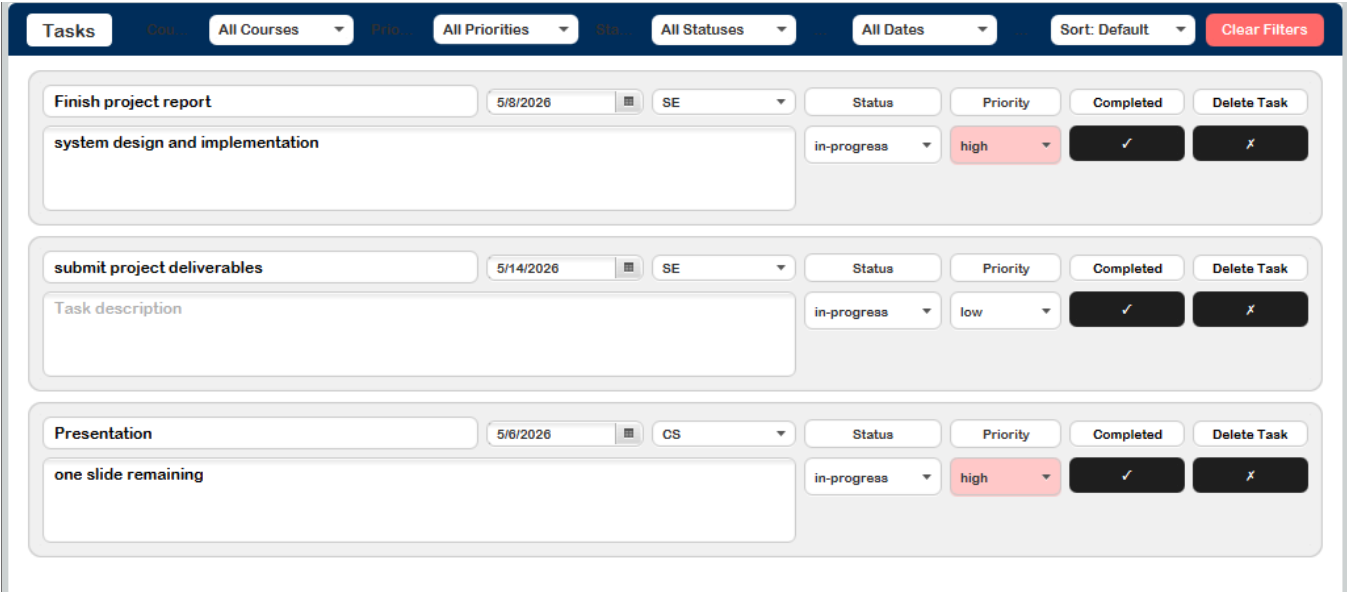


Figure 3.2a: list of tasks with no filters applied yet

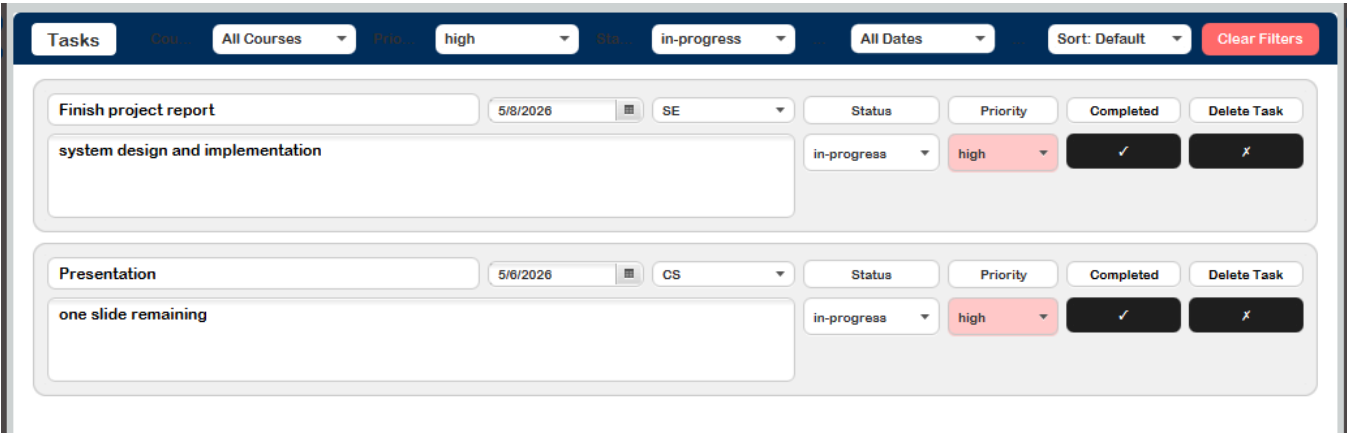


Figure 3.2b: ANDFilter applied, showing the list of tasks with priority and status filter applied

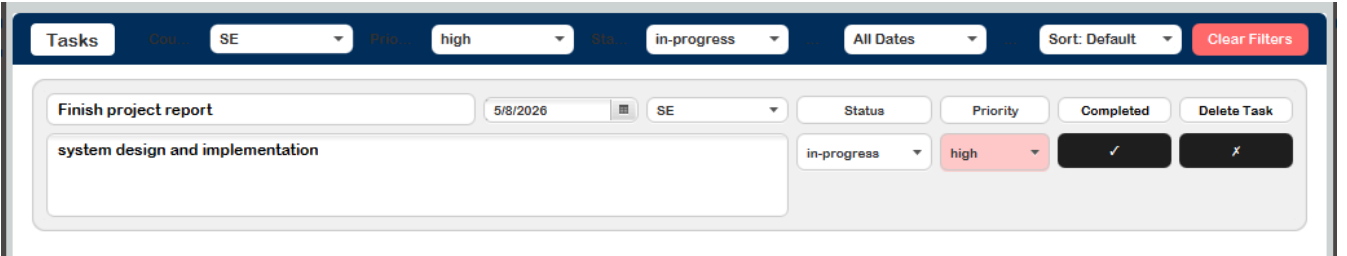


Figure 3.3c: ANDFilter applied; showing list of tasks with course, priority, and status filter applied

3.3 Cache Pattern

The Cache system was integrated into CanvasService, the class responsible for all data retrieval. Each of the three main public fetch methods (fetchCourses, fetchAssignments, fetchAnnouncements) follows the same three-layer sequence mentioned above. On every call, CanvasService first requests CacheManager using a string key. Since courses do not change within a session, the key is simply “courses”. If the memory cache holds a valid entry, it is returned immediately with no further work. On a cache miss, the CSV repository is requested. If data is found, it is written back into the memory cache before being returned, so the next access is served from the memory instead. If the repository is also missing, the Canvas API is called. Results are then updated to the CSV repository and written into the memory cache. The default of maxSize = 12 per cache was chosen arbitrarily for demonstration purposes.

3.4 GUI Tasks

The Tasks tab provides the user with a complete task management workflow. Clicking “Add Task” opens a pop-up window where the user may fill in a title, description, due date, course, completion status, and a priority level. On save, the new task is immediately displayed as a new row in the task list. Each task row also supports editing directly within the row after the task has been created. The user may revise the title and description by typing directly into their respective text fields, select a new due date using the date picker, and update the status, course, and priority using the dropdown menus provided. All changes are saved automatically as the user makes them. Additionally, priority level is given immediate visual feedback through color-coded styling on the priority dropdown, where high priority displays red, medium displays yellow, and low displays white. Furthermore, completing a task grays out its card and title and relabels the button to "Undo," giving the user a clear visual indication of the task's completion state. The user may also delete tasks individually or all at once using the "Delete All" option, and actions such as creating, deleting, and toggling completion are fully undoable and redoable through the undo and redo buttons. Additionally, all tasks persist across sessions, meaning the user's task list is retained even after closing and reopening the application. Lastly, the filter bar allows the user to narrow the task list by course, status, priority, and due date, and the sort controls allow the user to order tasks by earliest or latest due date, with undated tasks always pushed to the bottom.

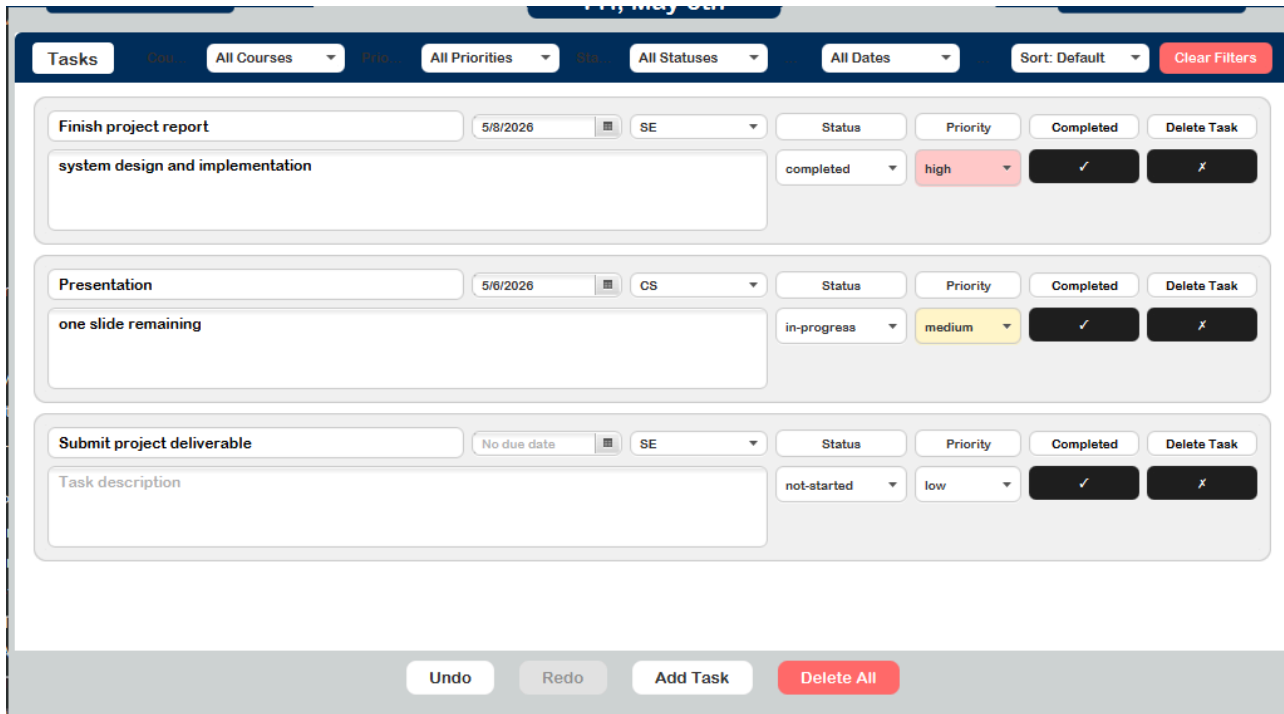


Figure 3.4a: example of complete task management workflow

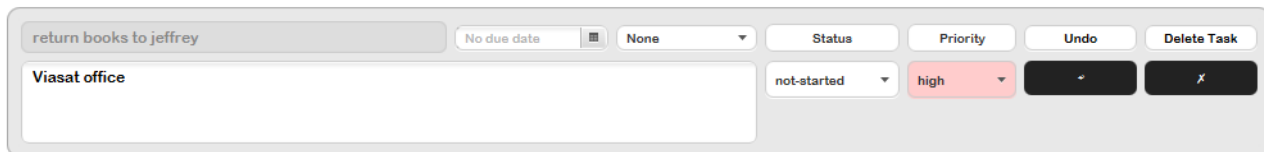


Figure 3.4b: a completed task, revealing the button to “Undo”

3.5 GUI Settings

The settings tab is designed to give the user a one stop shop for all of the different views, settings, and custom themes they may or may not want to change. You have the ability to switch to and from different views like the Announcements, Assignments or Tasks panel. You can change whether or not you prefer to view Announcements or Assignments in a dedicated day view, or the default week view. Once you have those setup, you can go ahead and change the day you want the week to start on if you don't like what you chose during the login processes. And then finally, you can choose between 3 themes; CougarPlanner the default theme, Dark which is a dark mode, and Sunset which is a pink flavored theme. All of the settings load instantly and do not require hitting a save button of

any kind, so a x button on the top right is provided to exit the settings panel at the user's convenience. And all settings persist after the user exits the program, so once they have a setup that suits their specific workflow, they never have to go into the settings again, besides switching up the view when needed.

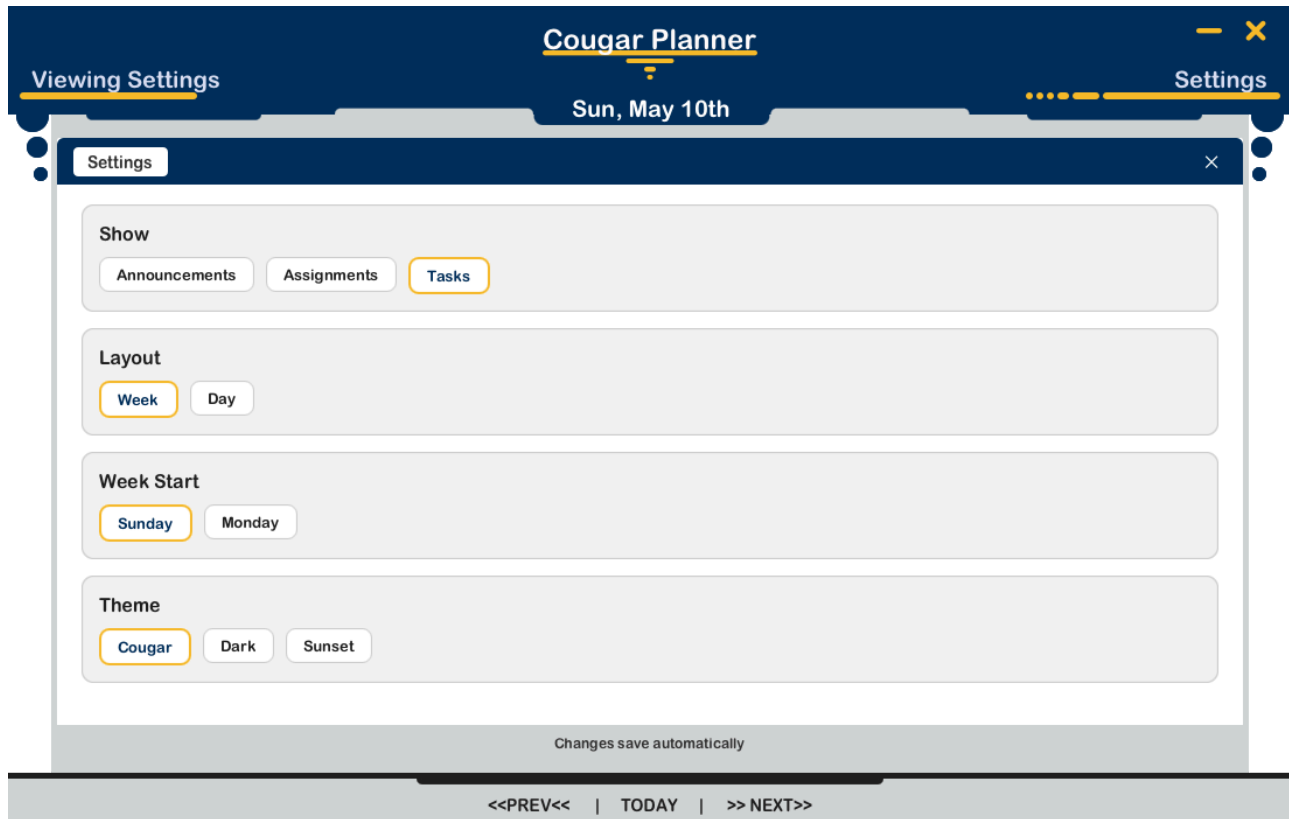


Figure 3.5a: Settings panel showing available settings

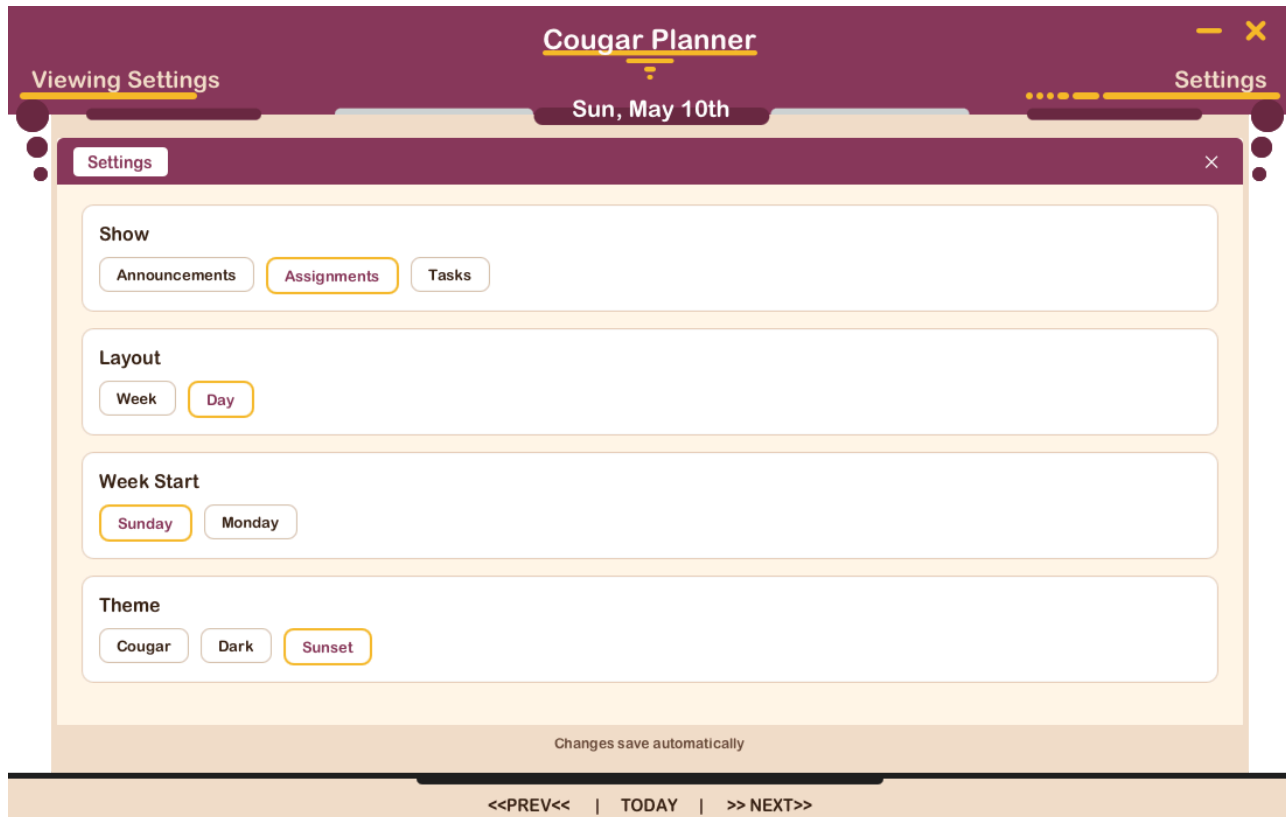


Figure 3.5b: Figure showing Sunset theme enabled through settings panel

5 Lessons Learned

5.1 Kenny

Creating the Cache Pattern for this project was an enjoyable learning experience. Since I have worked on this project before, mainly on the controllers and event handling within the GUI, I thought that working more on the backend side was a good change of pace. Even though we couldn't see insane efficiency in our project due to implementing this cache system, I still think that creating it had its benefits. Cache systems are necessary for nearly all applications dealing with larger amounts of data because it generally speeds data retrieval immensely. While I implemented a LRU style cache, doing so has made me more interested to see different types of caches and compare their efficiency, structure, and overall design. I also am sure creating cache systems varies drastically when looking at different programming languages or even machine level code. Working

in this group was quite seamless, we delegated our work neatly and met our deadlines with no problems. Overall I feel like this project was a complete success.

5.2 Amanda

My main role in this project focused on implementing the Tasks GUI. This was my first project-level experience implementing a GUI and working with JavaFX and SceneBuilder. Prior to this, I had only completed a few basic exercises through YouTube tutorials. Through this project, I gained hands-on experience wiring FXML controllers together, working with JavaFX components such as the date picker, dropdown menus, and text fields, and passing data between controllers using callbacks. I thoroughly enjoyed the instant feedback and seeing the program brought to life, and I found it rewarding to embrace my creative side throughout the process. I am very proud of how the Tasks feature turned out overall. In regards to the team aspect, I have worked with this team before and genuinely enjoyed collaborating with them again. I think we have only grown stronger as a team over time and have become even more efficient, productive, and communicative. Overall, my key lessons learned from this project are centered around developing new technical skills in JavaFX and applying the design patterns we have studied throughout the course in a meaningful, real-world context.

5.3 Maria

Working on this project taught me a lot about how to work inside an existing system instead of starting everything from scratch. At first, it took some trial and error to understand how the different parts of the codebase connected, especially when I started integrating the Command Pattern into the tasks feature with a different Java IDE that I have never used (Intelij). I had to spend time reading through the existing controller, repository logic, and established comments before I felt confident adding new code without breaking anything. Once I got comfortable with the structure, implementing the new commands became much more straightforward, and I started to appreciate how the design patterns kept everything organized. Overall, this was a really valuable experience for me because it was one of the first times I worked on a system I didn't originally build. It helped me get better at reading other people's code, understanding architecture decisions, and contributing new features in a way that fits the existing design.

5.4 Manal

Looking back at the whole project and what I learned in software architecture class, the biggest takeaway for me is that design patterns are tools, not rules. The trick is knowing when a pattern actually solves a problem versus when it just makes the code harder to follow. I created a TaskFilter interface with one method, then I made four different filters. Each one works completely differently inside, but because they all follow the same interface, the rest of the code does not care. The Controller just calls filter() and gets back a filtered list. This made so much sense to me, keeping things simple and confident. I also learned about separating responsibilities. The Filter Controller handles all the UI stuff, but the actual filter classes don't know anything about buttons or screens. This separation meant I could change how filters work without messing up the UI, and vice versa. Finally, I learned that good architecture saves time later. When new requirements come in, the separation of concerns and loose coupling make it a little easier to refactor the code.

5.5 Daniel

For me, learning how to use JavaFX was a completely new experience for me, and how it interacts with the controllers and other layers of the project. It was really hard to wrap my head around at first, seeing as I never touched that side of the project the first go around we had for this. However, with my team mates being able to clear things up, having access to youtube for tutorials, AI in case I had any questions that were hard to answer/not obvious, and of course SceneBuilder. It all made creating a modern GUI in JavaFX 10x easier and I was able to complete the goals we had set for the settings panel. In particular, making sure that theme system which uses dedicated css files we load dynamically, was a particular challenge. At first we had everything loaded manually in terms of colors using inline css, but after learning how to leverage JavaFX to use different css files, I was able to get it to work and switch as we need it during runtime. I came away pretty happy with the particular parts I was assigned to because of that, and now am much more confident with JavaFX and GUI's in general.

Works Cited

- Bro Code. "JavaFX DatePicker." YouTube, 3 Mar. 2021,
<https://www.youtube.com/watch?v=wxhGKR3PQpo>.
- Bro Code. "JavaFX GridPane." YouTube, 19 Mar. 2021,
<https://www.youtube.com/watch?v=InjJ8LcPqck>.
- Bro Code. "JavaFX ListView." YouTube, n.d.,
<https://www.youtube.com/watch?v=gLr5RcWz6x8>.
- Bro Code. "JavaFX communication between controllers." YouTube, n.d.,
<https://www.youtube.com/watch?v=Pqfd4hoi5cc>.
- Bro Code. "JavaFX DatePicker." YouTube, 3 Mar. 2021,
<https://www.youtube.com/watch?v=3Ht-JMQh2JI>.
- "Command." Refactoring.Guru, refactoring.guru/design-patterns/command. Accessed 1 May 2026.
- "Command Pattern." GeeksforGeeks,
www.geeksforgeeks.org/system-design/command-pattern. Accessed 1 May 2026.
- Deimos Coding Projects. "How to Fix JavaFX FXML Loader Crashing General Exception Errors." YouTube, n.d., <https://www.youtube.com/watch?v=fl6Wp1I9wHQ>.
- Genuine Coder. "JavaFX : Loading New FXML in the Same Scene / Window." YouTube, 27 Feb. 2020, <https://www.youtube.com/watch?v=fbkJdXF7EQM>.
- Genuine Coder. "JavaFX Scene Builder Tutorial - Make GUI Responsive." YouTube, n.d.,
<https://www.youtube.com/watch?v=WDaXpDtYk3E>.

Mistry, Aneesh. "New to Java 8: Functional Interfaces (Function, Consumer, Predicate, Supplier) - Tutorial." YouTube, n.d.,

<https://www.youtube.com/watch?v=PPwVwpdYFeU>.

Noble Code Monkeys. "JavaFX Tutorial - Toggle Button and Checkbox Click Event Selection." YouTube, n.d., https://www.youtube.com/watch?v=hwCbXOM4_Qc.

Random code. "JavaFX and Scene Builder Course - IntelliJ #22: How to change stages from inside the controller." YouTube, n.d.,

<https://www.youtube.com/watch?v=ZiDg1kiVDI8>.

Random code. "How to setup JavaFX 11+ with IntelliJ on Windows [2022]." YouTube, n.d., <https://www.youtube.com/watch?v=RJOza3XQk34>.

Random code. "JavaFX and Scene Builder - ComboBox." YouTube, n.d.,

<https://www.youtube.com/watch?v=0zcnv4hVvVw>.

Ripples Code. "JAVAFx Tutorial 16-SceneBuilder DatePicker." YouTube, n.d.,

<https://www.youtube.com/watch?v=bdAMljdPj44>.

Software Development Tutorials. "JavaFX with Scene Builder : Lecture 2: Buttons, load Main Items FXML document in main Layout FXML." YouTube, n.d.,

<https://www.youtube.com/watch?v=cS6JsmECDcU>.